Polytechnique Montréal

Assignment #1

Machine Learning

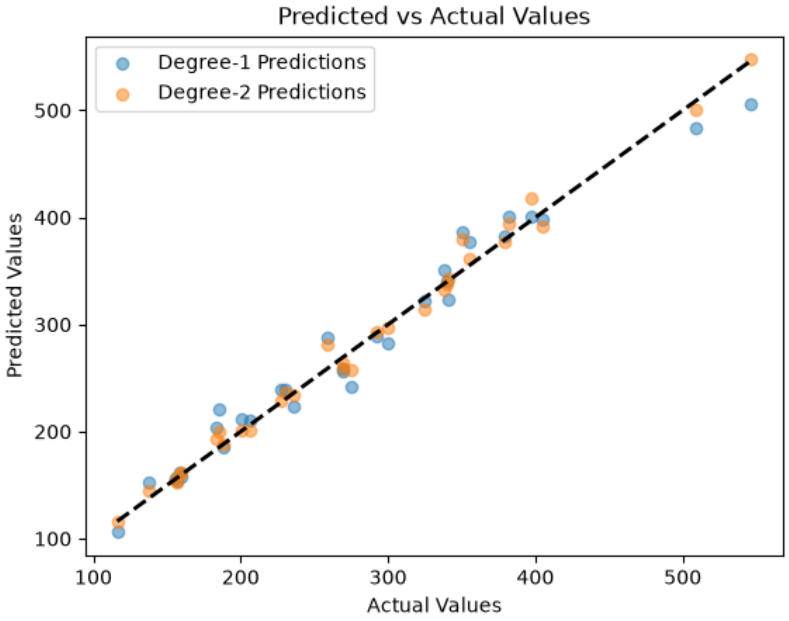
Course: INF8245AE – Machine Learning
Term: Fall 2026

Student name: Matthias BENOIST
Student ID: 2487407
Email: matthias-alain-edouard.benoist@etud.polymtl.ca

Question 2: Feature Transformations and Least-Squares Regression

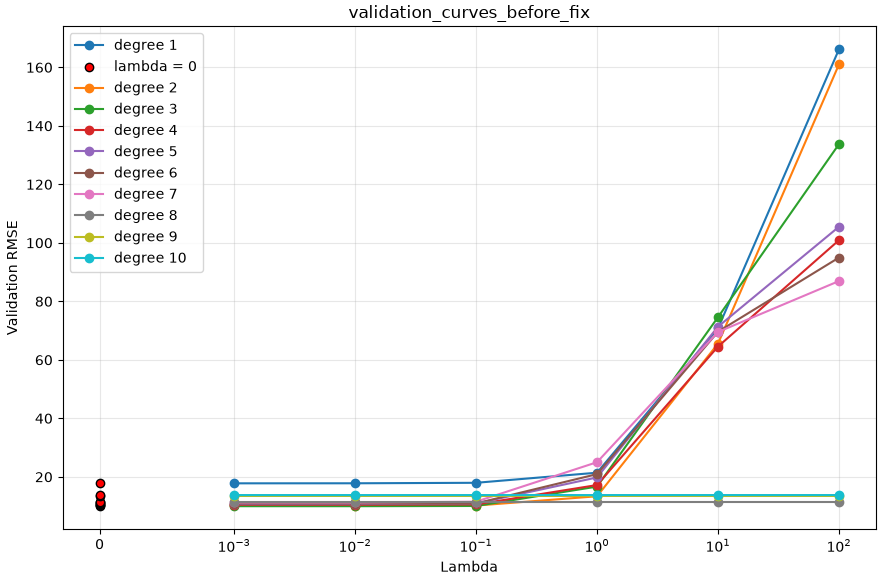
Model	Validation RMSE
Degree-1 least squares	17.7839
Degree-2 least squares	10.2109

The degree-2 model has a lower validation RMSE than the degree-1 model, indicating that the target has a nonlinear relationship with at least one of the five raw features that a purely linear model cannot capture.



Question 4: Debugging the Supplied Ridge Implementation

Before the fix



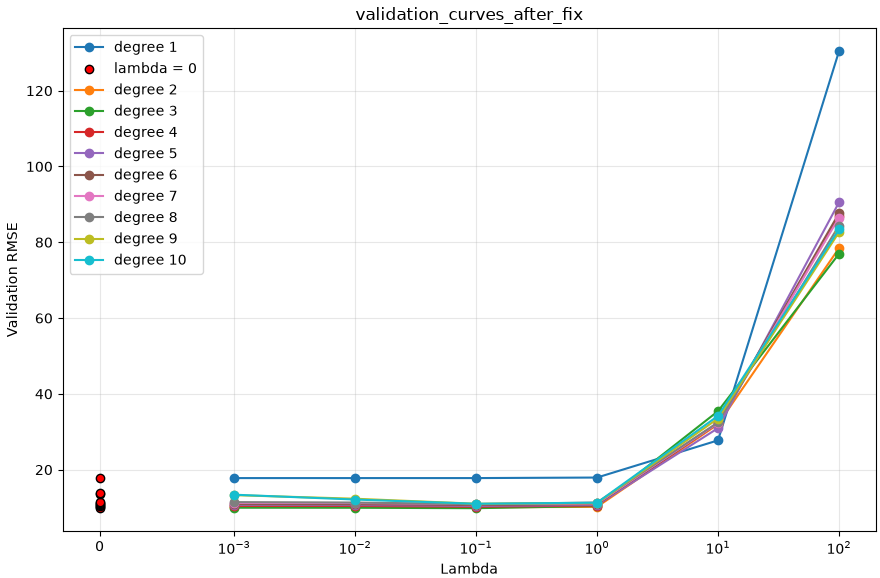
With the supplied `ridge_fit`, the normal equations matrix is built as

$$X.T@X + lambda \times np.ones((d, d))$$

λ is added to every entry of a $d \times d$ matrix of ones, rather than only to the diagonal. This does not implement the ridge penalty $\lambda\|w\|_2^2$ at all: the true ridge solution minimizes $\|Xw - y\|_2^2 + \lambda\|w\|_2^2$, whose normal equations are $A = X^\top X + \lambda I$.

The effect is visible in the plot: several degrees (8, 9, and 10) are almost flat across the entire range of λ.

After the fix



Correcting the function restores the intended ridge penalty. We do not see a fully U-shaped tradeoff but the curve goes down a little at the beginning and then goes up by much larger steps at the end. A small λ does not hurt and can even help the performance but a high one will worsen the validation by a lot.

Question 5: Final Model Selection

Quantity	Value
Selected degree	3
Selected λ	0.1
Validation RMSE	9.8382
Test RMSE	9.5223
Test MAE	7.4130
Test R^2	0.9912

The test RMSE (9.52) is close to the validation RMSE (9.84) obtained during model selection.

Question 6: Mini-Batch Subgradient Descent for Lasso Regression

Note : This part was done by giving handwritten notes to Claude to generate the math symbols and translate it in English.

(a) Derivation

The lasso objective is

$$L(w) = \frac{1}{n}\|Xw - y\|_2^2 + \lambda\|w\|_1, \qquad \|w\|_1 = \sum_{j=1}^{d} |w_j|.$$

Gradient of the squared-error term. Writing $f(w) = \frac{1}{n}\|Xw - y\|_2^2$,

$$\nabla f(w) = \frac{2}{n}X^\top(Xw - y).$$

Derivative of $|w_j|$ for $w_j \neq 0$.

$$\frac{d}{dw_j}|w_j| = \begin{cases} +1 & w_j > 0 \\ -1 & w_j < 0. \end{cases} = \text{sign}(w_j)$$

Non-differentiability at $w_j = 0$. At $w_j = 0$, the left derivative of $|w_j|$ is -1 and the right derivative is $+1$; these disagree, so $|w_j|$ has a corner at the origin and no ordinary derivative exists there. Consequently $L(w)$ is not differentiable whenever any coordinate of w is exactly zero.

Subdifferential of $|w_j|$ at $w_j = 0$. A scalar g_j is a subgradient of $\phi(w_j) = |w_j|$ at $w_j = 0$ if

$$|u| \;\geq\; g_j\, u \qquad \text{for all } u \in \mathbb{R}.$$

We determine the set of valid g_j by considering three cases for u:

- $u = 0$: the inequality reads $0 \geq 0$, which always holds.
- $u > 0$: dividing by u gives $1 \geq g_j$, i.e. $g_j \in]-\infty, 1]$.
- $u < 0$: dividing by u (which flips the inequality) gives $-1 \leq g_j$, i.e. $g_j \in [-1, +\infty[$.

Intersecting the three cases, g_j must satisfy all of them simultaneously, so

$$\partial |w_j| \big|_{w_j=0} = [-1, 1].$$

Subgradient of the full lasso objective. Since the squared-error term is differentiable everywhere, a valid subgradient of L is obtained by adding its gradient to any choice of subgradient g_j of $\lambda|w_j|$, coordinate by coordinate. Let $g = (g_1, \ldots, g_d)$ collect these coordinate-wise subgradients, with

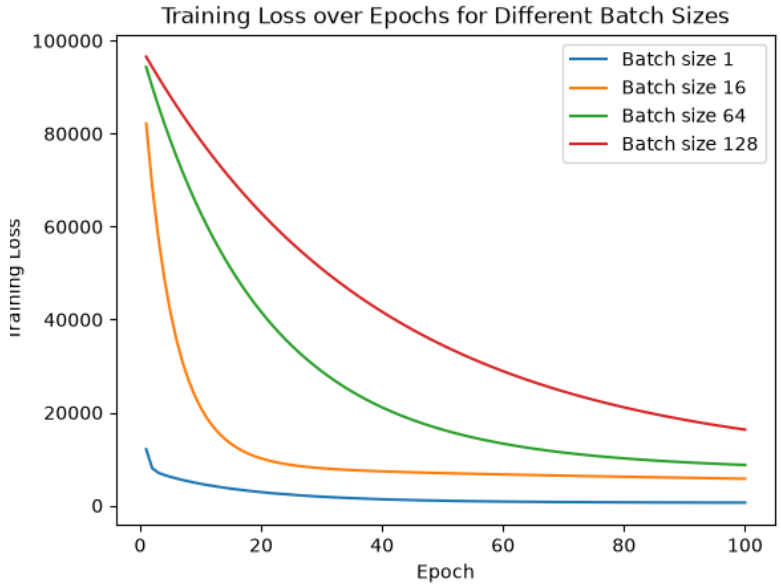
$$g_j = \begin{cases} \lambda \operatorname{sign}(w_j) & w_j \neq 0 \\ 0 & w_j = 0. \end{cases}$$

Then a subgradient of the full lasso objective $L(w)$ is

$$\nabla_w L(w) = \frac{2}{n} X^\top (Xw - y) \;+\; g,$$

which is exactly what `lasso_gradient` computes using `np.sign(w)` (with `np.sign(0)=0` giving the chosen $g_j = 0$ at zero coordinates).

Question 7: Effect of Batch Size

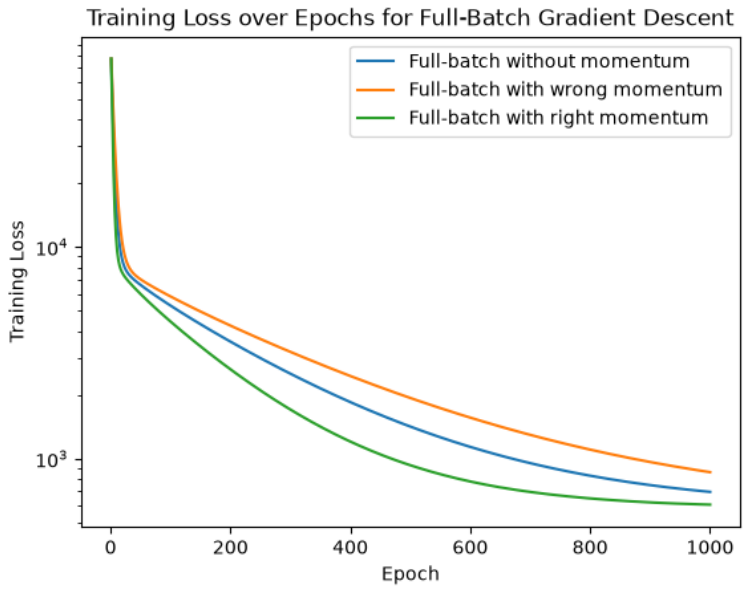


Batch size	Validation RMSE
1	10.72
16	68.24
64	80.77
128 (full batch)	111.39

With $n = 128$ training points, a batch size of B produces $\lceil n/B \rceil$ mini-batches, and hence $\lceil n/B \rceil$ parameter updates per epoch: batch size 1 gives 128 updates/epoch, batch size 16 gives 8 updates/epoch, batch size 64 gives 2 updates/epoch, and full-batch (128) gives exactly 1 update/epoch.

This directly explains the training-loss curves: with only 100 epochs total, batch size 1 performs 12,800 updates and has essentially converged, giving both the lowest training loss and the lowest validation RMSE. Batch size 128 performs only 100 updates total and is still early in optimization after 100 epochs, which is why its training loss curve is still visibly decreasing and its validation RMSE is by far the worst.

Question 8: Debugging Momentum



Unexpected effect of the supplied momentum update. Momentum is supposed to accelerate convergence relative to plain gradient descent, but the plot shows the opposite: "full-batch with wrong momentum" (orange) converges more slowly than plain gradient descent (blue) throughout training, ending at a higher loss after 1000 epochs.

The bug. The supplied update is

$$v \leftarrow -\beta v - \eta g, \qquad w \leftarrow w + v.$$

The correct momentum update should accumulate previous velocity with the same sign it already has so that consistent gradient directions reinforce each other across iterations.

The fix. Changing the update to

$$v \leftarrow \beta v - \eta g, \qquad w \leftarrow w + v,$$